

● SPOT THE BUG

One of these lines is a SQL injection.

Real WordPress core. The wp2shell chain,
CVE-2026-60137.

- THE CODE THAT SHIPPED

WordPress 7.0.1, WP_Query.

...

```
if ( is_array( $v['author__not_in'] ) ) {  
    $v['author__not_in'] =  
        array_map( 'absint', $v['author__not_in'] );  
}  
$out = implode( ',', (array) $v['author__not_in'] );  
$where .= " ... post_author NOT IN ($out) ";
```

Handling the "exclude these authors" filter. Where
is the bug?

(\$v = \$query_vars, simplified to fit)

- THE BUG

Sanitization behind a type check.

```
...  
if ( is_array( $v['author__not_in'] ) ) {  
    $v['author__not_in'] =  
        array_map( 'absint', $v['author__not_in'] );  
}  
$out = implode( ',', (array) $v['author__not_in'] );  
$where .= " ... post_author NOT IN ($out) ";
```

absint runs only inside is_array(). Pass author__not_in as a STRING and the whole branch is skipped.

- WHERE IT LANDS

Straight into the query.

...

```
if ( is_array( $v['author__not_in'] ) ) {  
    $v['author__not_in'] =  
        array_map( 'absint', $v['author__not_in'] );  
}  
$out = implode( ',', (array) $v['author__not_in'] );  
$where .= " ... post_author NOT IN ($out) ";
```

A string is passed through unchanged and concatenated into the SQL WHERE clause. That is the injection.

● THE TELL

Its own neighbour did it right.

...

```
// three lines below, for author__in:
```

```
$in = implode( ',',
```

```
    array_map( 'absint',
```

```
        (array) $v['author__in'] ) );
```

`author__in` casts and sanitizes for EVERY type, unconditionally. Two sibling branches, same job, one guarded and one not.

- THE FIX

7.0.2: coerce, always.

```
...  
$ids = wp_parse_id_list(  
    $v['author__not_in'] );  
$out = implode( ',', $ids );  
$where .= " ... NOT IN ($out) ";
```

`wp_parse_id_list()` returns a list of integers for a string OR an array. No branch left to skip. One function, whole class of bug gone.

- FULL BREAKDOWN + SAFE LAB

Reproduce it. Detect it. Fix it.

Real source diff, the batch bug it chains with, detection rules, and a Docker lab that proves it safely.

[zenithgenius.github.io/
wordpress-batch-rce-lab](https://zenithgenius.github.io/wordpress-batch-rce-lab)